Embedded Systems Testing Project Requirement

RTOS-Based Multi-Sensor Embedded System on ESP Board

1. Project Goal

Each team must design, implement, and test an **RTOS-based embedded system** using an ESP board. The project must prove that the team can apply embedded testing concepts, not only write working firmware.
The system must use:

- ESP32 or ESP8266 board
- RTOS as the main software architecture
- At least **3 sensors**
- At least one **analog input**
- At least one **digital input/output**
- At least one **communication-based sensor/interface** such as I2C, SPI, UART, WiFi, Bluetooth, or MQTT

The project must demonstrate **functional testing, integration testing, RTOS testing, timing analysis, synchronization testing, stress testing, fault injection, and test automation**.
This follows the course focus on embedded systems as hardware/software systems where testing must verify behavior, timing, power/resource use, hardware interaction, robustness, and reliability.

---

2. Required System Architecture

Students must divide the firmware into RTOS tasks. A single `while(1)` loop solution is not accepted.
Minimum required tasks:

1. **Analog Sensor Task**
   Reads an analog sensor periodically using ADC.
2. **Digital Sensor/Input Task**
   Reads a digital sensor, button, switch, PIR, IR sensor, ultrasonic trigger/echo, or similar.
3. **Communication Sensor Task**
   Reads data from an I2C, SPI, UART, WiFi, Bluetooth, or MQTT-based device.
4. **Processing / Decision Task**
   Receives sensor data, validates it, filters it, and makes system decisions.
5. **Output / Actuator Task**
   Controls LED, buzzer, relay, motor, display, or another output.
6. **Logging / Diagnostic Task**
   Prints test logs, timing measurements, queue status, errors, and system state.

Students must use at least:

- One **queue**
- One **semaphore**
- One **mutex**
- One timeout-based RTOS operation
- One ISR or interrupt-driven input if hardware allows

RTOS testing must verify task priorities, blocking, wake-up behavior, deadlines, shared resources, and stability under load.

---

3. Sensor Requirement

Each team must choose 3 sensors from different categories.
Example acceptable combinations:

Category	Example
Analog input	LDR, potentiometer, LM35, soil moisture analog output
Digital input	PIR sensor, button, flame sensor digital output, IR obstacle sensor
Communication sensor	DHT via single-wire, MPU6050 via I2C, BME280 via I2C/SPI, GPS via UART, RFID via SPI

The team must explain:

- Sensor purpose
- Interface type
- Sampling period
- Expected normal range
- Invalid range
- Failure behavior
- Test cases for each sensor

---

4. Mandatory RTOS Concepts to Demonstrate

The project must include and test the following concepts:

A. Task Scheduling

Students must define:

Task	Priority	Period	Deadline	Purpose
Analog sensor task				
Digital sensor task				
Communication task				
Processing task				
Output task				
Logging task				

They must justify why each priority was chosen.

They must measure:

- Task period accuracy
- Jitter
- Missed deadlines
- Worst-case execution time

Real-time testing must focus on worst-case behavior, not only average behavior.

B. Queue-Based Communication

At least one sensor task must send data to another task using an RTOS queue.

Students must test:

- Normal queue operation
- Queue full condition
- Producer faster than consumer
- Consumer delay
- Queue overflow recovery
- Maximum queue occupancy
- Dropped message count

Queues must be tested under burst traffic, stress conditions, long-duration execution, overflow verification, and timing analysis.

C. Semaphore Synchronization

Students must use a binary semaphore for event signaling.

Example:

- ISR gives semaphore
- Sensor task waits for semaphore
- Button interrupt wakes task
- Timer interrupt triggers periodic acquisition

Students must test:

- Missed signal detection

- Wake-up latency
- Timeout behavior
- Semaphore behavior under CPU load
- ISR-to-task synchronization

Semaphore testing must verify missed signals, wake-up timing, timeout handling, ISR synchronization, and blocking correctness.

D. Mutex Protection

Students must use a mutex to protect a shared resource.

Acceptable shared resources:

- UART serial printing
- I2C bus
- SPI bus
- Shared data structure
- Display
- Log buffer

Students must test:

- Correct ownership
- Blocking time
- Mutex hold time
- Shared data corruption without mutex
- Correct behavior with mutex
- Priority inversion scenario if possible

Mutexes protect shared resources, have ownership, and may support priority inheritance, so testing must check blocking timing, ownership, deadlock behavior, and priority inversion.

---

5. Required Testing Workflow

Students must follow this workflow.

Phase 1 — Requirement Definition

Students must write at least 10 clear requirements.

Example:

ID	Requirement	Type
REQ-01	Analog sensor shall be sampled every 100 ms	Real-time
REQ-02	System shall detect invalid sensor readings	Functional
REQ-03	Queue shall not lose data under normal load	RTOS
REQ-04	Processing task shall respond within 200 ms	Timing
REQ-05	System shall recover from communication sensor failure	Fault tolerance

Each requirement must have at least one test case.

Traceability is mandatory because embedded testing requires linking each requirement to one or more verification tests.

Phase 2 — Unit Testing

Students must unit test at least 3 software modules.

Required unit-tested modules:

1. Sensor value validation function
2. Decision logic function
3. Data filtering or conversion function

Example functions:

```
bool isTemperatureValid(float temp);
int convertAdcToPercent(int adc);
SystemState decideSystemState(SensorData data);
```

Students may use:

- Unity
- Ceedling
- CppUTest
- GoogleTest
- Arduino host test
- Manual test harness

Unit testing must use mocks/stubs when hardware is not required. Unit testing is expected to verify isolated logic before hardware integration.

Phase 3 — Integration Testing

Students must integrate the system step by step.

Required integration order:

1. Test each sensor alone
2. Test each sensor with its RTOS task
3. Test sensor task with queue
4. Test queue with processing task
5. Test processing task with output task
6. Test complete system
7. Test complete system under load

Students must document one integration bug and how they found it.

Incremental integration is required because embedded failures often appear in the interaction between modules, drivers, peripherals, interrupts, and timing.

Phase 4 — System Testing

Students must test the full system against all requirements.

System testing must include:

- Normal operation
- Boundary values
- Invalid sensor values
- Sensor disconnect or stuck value
- Queue overflow
- High CPU load
- Long-duration run
- Reset/recovery behavior

Phase 5 — RTOS and Timing Testing

Students must measure and report:

Metric	Required
WCET of each task	Yes
Task period jitter	Yes
Queue latency	Yes
Semaphore wake-up delay	Yes
Mutex blocking time	Yes
Missed deadline count	Yes
CPU load estimate	Yes
Stack high-water mark if available	Bonus

Timing may be measured using:

- `micros()`
- ESP timer
- FreeRTOS runtime stats
- GPIO toggle + oscilloscope/logic analyzer
- Serial timestamp logs

Students must prove that the system works under worst-case timing conditions, because average timing alone is not sufficient for real-time systems.

---

6. Mandatory Fault Injection Tests

Each team must intentionally inject faults.
Minimum required fault tests:

Fault	Expected Student Action
Analog sensor out of range	Detect and enter safe state
Digital sensor stuck high/low	Detect abnormal behavior if possible
Communication sensor disconnected	Timeout and error reporting
Queue overflow	Count dropped data and recover
Delayed consumer task	Show queue buildup
Mutex held too long	Measure blocking impact
Artificial CPU load	Show timing/jitter impact
Interrupt or semaphore event burst	Check missed wakeups

Fault injection is mandatory because real embedded systems must survive abnormal conditions such as missing sensor data, communication failure, queue overflow, timing disturbance, and corrupted inputs.

---

7. Stress Testing Requirement

Students must run a stress test for at least **10 minutes**.
During stress testing, they must create at least 3 of the following:

- Faster sensor sampling
- Artificial CPU load task
- Queue flooding
- Repeated interrupts
- Random task delay
- Communication burst
- Serial logging burst
- Delayed processing task

Students must record:

- Maximum queue occupancy
- Dropped samples
- Maximum task execution time
- Worst jitter
- Missed deadlines
- System reset or watchdog events

Stress testing is required because race conditions, deadlocks, queue overflow, and timing failures may not appear during simple classroom demonstrations.

---

8. Race Condition Demonstration

Students must intentionally create a small unsafe shared variable example.
Example:

```
volatile int sharedCounter = 0;
```

Two tasks update it without protection, then repeat the test using a mutex or atomic protection.

They must show:

- Incorrect result without protection
- Correct result with protection
- Explanation of why the race occurred

Race conditions are dangerous because system behavior depends on unpredictable execution timing, task switching, interrupt timing, and CPU load.

---

9. Deadlock or Priority Inversion Demonstration

Students must demonstrate **one** of the following.

Option A — Deadlock Demonstration

Create two tasks and two mutexes.

- Task A locks UART then SPI
- Task B locks SPI then UART

Then fix it using:

- Fixed lock ordering
- Timeout-based locking
- Avoiding nested locks

Option B — Priority Inversion Demonstration

Create:

- Low-priority task holding mutex
- High-priority task waiting for mutex
- Medium-priority task consuming CPU

Then observe the delay and explain priority inheritance if supported.

Deadlock and priority inversion are required topics because they are common RTOS synchronization failures that can cause missed deadlines, frozen systems, or watchdog resets.

---

10. Test Automation Bonus

Bonus marks may be awarded for automating the testing flow.

Students may use:

- Python script reading serial output
- Pytest
- Robot Framework
- PlatformIO test
- GitHub Actions build
- Serial log parser
- CSV report generator
- Automatic pass/fail test runner

Example automation flow:

1. Flash firmware
2. Open serial port
3. Send test commands
4. Collect logs
5. Check pass/fail conditions
6. Generate test report

Bonus tests:

- Automatic queue overflow test
- Automatic timing log parser
- Automatic requirement coverage table
- Automatic fault injection command script
- CI build + unit tests

Embedded CI and regression automation are valuable because every change should be automatically built and tested to catch regressions early.

11. Required Deliverables

Each team must submit:

1. **Source code**
2. **Block diagram**
3. **RTOS task architecture table**
4. **Sensor interface table**
5. **Requirement list**
6. **Requirement-to-test traceability matrix**
7. **Unit test results**
8. **Integration test results**
9. **System test results**
10. **Fault injection results**
11. **Stress test report**
12. **Timing measurement report**
13. **Short demo video**
14. **Final presentation**

12. Traceability Matrix Template

Requirement ID	Requirement	Test ID	Test Type	Result
REQ-01	Analog sensor sampled every 100 ms	TC-01	Timing	Pass/Fail
REQ-02	Queue shall not overflow under normal load	TC-02	RTOS	Pass/Fail
REQ-03	Sensor failure shall be detected	TC-03	Fault injection	Pass/Fail
REQ-04	Output shall activate when threshold exceeded	TC-04	Functional	Pass/Fail
REQ-05	Mutex shall prevent shared UART corruption	TC-05	Synchronization	Pass/Fail

13. Grading Rubric

Category	Marks
RTOS architecture and task design	15
Sensor integration and hardware correctness	10
Queue, semaphore, mutex usage	15
Unit and integration testing	15
Timing, jitter, WCET, deadline analysis	15
Fault injection and stress testing	15
Documentation and traceability	10
Demo quality	5
Test automation bonus	+10

14. Minimum Acceptance Criteria

A project will not pass unless it includes:

- RTOS-based design
- 3 sensors
- Analog input
- Digital input/output
- Communication interface
- Queue
- Semaphore
- Mutex

- At least 10 requirements
- At least 10 test cases
- Fault injection
- Stress test
- Timing measurement
- Final report with evidence